

فایل محور نهایی PostgreSQL DDL

اپلیکیشن فروش محصولات پلاستیکی

:این بسته شامل سه فایل است

- `schema.sql`
- `seed.sql`
- `down.sql`

:ترتیب اجرا

1. `schema.sql`
2. `seed.sql`
3. `down.sql`: در صورت نیاز برای حذف کامل

1) `schema.sql`

```
-- =====
-- schema.sql
-- PostgreSQL DDL for Plastic Products Sales Application
-- =====
```

```
BEGIN;
```

```
-- -----
-- Extension
-- -----
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- -----
-- ENUM Types
-- -----
```

```

CREATE TYPE user_role_enum AS ENUM ('admin', 'buyer', 'visitor');
CREATE TYPE order_status_enum AS ENUM ('pending', 'assigned', 'loading',
'delivered', 'cancelled');
CREATE TYPE product_quality_enum AS ENUM ('اوليه', 'بازيافتی');
CREATE TYPE otp_purpose_enum AS ENUM ('register', 'login',
'change_phone');
CREATE TYPE deletion_status_enum AS ENUM ('pending', 'approved',
'rejected');
CREATE TYPE stock_reason_enum AS ENUM ('initial', 'sale', 'restock',
'adjustment');
CREATE TYPE notification_type_enum AS ENUM ('sms', 'push');

```

```

-- -----
-- Base Tables
-- -----

```

```

CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    phone VARCHAR(11) NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    address TEXT,
    role user_role_enum NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE
);

```

```

CREATE TABLE system_settings (
    setting_key VARCHAR(100) PRIMARY KEY,
    setting_value TEXT NOT NULL,
    description TEXT,
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

```

```

-- -----
-- Core Business Tables
-- -----

```

```

CREATE TABLE products (
  id BIGSERIAL PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  price DECIMAL(15,2) NOT NULL,
  weight DECIMAL(10,2) NOT NULL,
  color VARCHAR(50),
  quality product_quality_enum NOT NULL,
  description TEXT,
  image_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
  stock INTEGER NOT NULL DEFAULT 0,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_by BIGINT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE,

  CONSTRAINT chk_products_price_nonnegative CHECK (price >= 0),
  CONSTRAINT chk_products_stock_nonnegative CHECK (stock >= 0),
  CONSTRAINT chk_products_image_urls_max_5 CHECK (
    CASE
      WHEN jsonb_typeof(image_urls) = 'array'
      THEN jsonb_array_length(image_urls) <= 5
      ELSE FALSE
    END
  ),
  CONSTRAINT fk_products_created_by
    FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE
    RESTRICT
);

```

```

CREATE TABLE otp_codes (
  id BIGSERIAL PRIMARY KEY,
  phone VARCHAR(11) NOT NULL,
  code VARCHAR(5) NOT NULL,
  purpose otp_purpose_enum NOT NULL,
  expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
  is_used BOOLEAN NOT NULL DEFAULT FALSE,

```

```
attempt_count INTEGER NOT NULL DEFAULT 0,  
locked_until TIMESTAMP WITH TIME ZONE,  
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
```

```
CONSTRAINT chk_otp_attempt_count_nonnegative CHECK (attempt_count  
>= 0)  
);
```

```
CREATE TABLE account_deletion_requests (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  status deletion_status_enum NOT NULL DEFAULT 'pending',  
  requested_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
  reviewed_by BIGINT,  
  reviewed_at TIMESTAMP WITH TIME ZONE,  
  admin_note TEXT,
```

```
CONSTRAINT fk_account_deletion_requests_user  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
CONSTRAINT fk_account_deletion_requests_reviewer  
  FOREIGN KEY (reviewed_by) REFERENCES users(id) ON DELETE SET  
NULL  
);
```

```
CREATE TABLE cart_items (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  product_id BIGINT NOT NULL,  
  quantity DECIMAL(10,2) NOT NULL,  
  added_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE,
```

```
CONSTRAINT chk_cart_items_quantity_positive CHECK (quantity > 0),  
CONSTRAINT fk_cart_items_user  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
CONSTRAINT fk_cart_items_product
```

```
        FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE  
CASCADE,  
        CONSTRAINT uq_cart_items_user_product UNIQUE (user_id, product_id)  
);
```

```
CREATE TABLE orders (  
    id BIGSERIAL PRIMARY KEY,  
    buyer_id BIGINT NOT NULL,  
    visitor_id BIGINT,  
    total_price DECIMAL(15,2) NOT NULL DEFAULT 0,  
    status_order_status_enum NOT NULL DEFAULT 'pending',  
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMP WITH TIME ZONE,  
  
    CONSTRAINT chk_orders_total_price_nonnegative CHECK (total_price >= 0),  
    CONSTRAINT fk_orders_buyer  
        FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE  
RESTRICT,  
    CONSTRAINT fk_orders_visitor  
        FOREIGN KEY (visitor_id) REFERENCES users(id) ON DELETE SET  
NULL  
);
```

```
CREATE TABLE order_items (  
    id BIGSERIAL PRIMARY KEY,  
    order_id BIGINT NOT NULL,  
    product_id BIGINT NOT NULL,  
    quantity DECIMAL(10,2) NOT NULL,  
    unit_price DECIMAL(15,2) NOT NULL,  
    total_price DECIMAL(15,2) GENERATED ALWAYS AS (quantity * unit_price)  
STORED,  
  
    CONSTRAINT chk_order_items_quantity_positive CHECK (quantity > 0),  
    CONSTRAINT chk_order_items_unit_price_nonnegative CHECK (unit_price  
>= 0),  
    CONSTRAINT fk_order_items_order
```

```
        FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE
CASCADE,
        CONSTRAINT fk_order_items_product
        FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE
RESTRICT
);
```

```
CREATE TABLE order_assignments (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL,
    old_visitor_id BIGINT,
    new_visitor_id BIGINT NOT NULL,
    assigned_by BIGINT NOT NULL,
    reason TEXT,
    assigned_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

    CONSTRAINT fk_order_assignments_order
        FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE
CASCADE,
    CONSTRAINT fk_order_assignments_old_visitor
        FOREIGN KEY (old_visitor_id) REFERENCES users(id) ON DELETE SET
NULL,
    CONSTRAINT fk_order_assignments_new_visitor
        FOREIGN KEY (new_visitor_id) REFERENCES users(id) ON DELETE
RESTRICT,
    CONSTRAINT fk_order_assignments_assigned_by
        FOREIGN KEY (assigned_by) REFERENCES users(id) ON DELETE
RESTRICT
);
```

```
CREATE TABLE order_status_histories (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL,
    old_status order_status_enum,
    new_status order_status_enum NOT NULL,
    changed_by BIGINT NOT NULL,
    note TEXT,
```

```
changed_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

CONSTRAINT fk_order_status_histories_order
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE
CASCADE,
CONSTRAINT fk_order_status_histories_changed_by
    FOREIGN KEY (changed_by) REFERENCES users(id) ON DELETE
RESTRICT
);
```

```
CREATE TABLE price_histories (
    id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL,
    old_price DECIMAL(15,2),
    new_price DECIMAL(15,2) NOT NULL,
    changed_by BIGINT NOT NULL,
    changed_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_price_histories_new_price_nonnegative CHECK
(new_price >= 0),
    CONSTRAINT fk_price_histories_product
        FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE
RESTRICT,
    CONSTRAINT fk_price_histories_changed_by
        FOREIGN KEY (changed_by) REFERENCES users(id) ON DELETE
RESTRICT
);
```

```
CREATE TABLE stock_histories (
    id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL,
    old_stock INTEGER,
    new_stock INTEGER NOT NULL,
    reason stock_reason_enum NOT NULL,
    changed_by BIGINT NOT NULL,
    changed_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
```

```

        CONSTRAINT chk_stock_histories_new_stock_nonnegative CHECK
(new_stock >= 0),
        CONSTRAINT fk_stock_histories_product
        FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE
RESTRICr,
        CONSTRAINT fk_stock_histories_changed_by
        FOREIGN KEY (changed_by) REFERENCES users(id) ON DELETE
RESTRICr
);

```

```

CREATE TABLE notifications (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL,
    related_type VARCHAR(30),
    related_id BIGINT,
    type notification_type_enum NOT NULL,
    message TEXT NOT NULL,
    sent_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    is_read BOOLEAN NOT NULL DEFAULT FALSE,

```

```

        CONSTRAINT fk_notifications_user
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

```

-- -----
-- Helper Functions
-- -----

```

```

CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE OR REPLACE FUNCTION recalculate_order_total(p_order_id BIGINT)

```

```
RETURNS VOID AS $$
```

```
BEGIN
```

```
    UPDATE orders
```

```
    SET total_price = COALESCE(
```

```
        (SELECT SUM(oi.total_price)
```

```
        FROM order_items oi
```

```
        WHERE oi.order_id = p_order_id),
```

```
        0
```

```
    ),
```

```
    updated_at = NOW()
```

```
    WHERE id = p_order_id;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION refresh_order_total_trigger()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF TG_OP = 'INSERT' THEN
```

```
        PERFORM recalculate_order_total(NEW.order_id);
```

```
        RETURN NEW;
```

```
    ELSIF TG_OP = 'DELETE' THEN
```

```
        PERFORM recalculate_order_total(OLD.order_id);
```

```
        RETURN OLD;
```

```
    ELSIF TG_OP = 'UPDATE' THEN
```

```
        PERFORM recalculate_order_total(OLD.order_id);
```

```
        IF NEW.order_id IS DISTINCT FROM OLD.order_id THEN
```

```
            PERFORM recalculate_order_total(NEW.order_id);
```

```
        END IF;
```

```
        RETURN NEW;
```

```
    END IF;
```

```
    RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION insert_initial_product_history()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO price_histories (product_id, old_price, new_price, changed_by,  
changed_at)  
    VALUES (NEW.id, NULL, NEW.price, NEW.created_by, NOW());  
  
    INSERT INTO stock_histories (product_id, old_stock, new_stock, reason,  
changed_by, changed_at)  
    VALUES (NEW.id, NULL, NEW.stock, 'initial', NEW.created_by, NOW());  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION sync_order_visitor()  
RETURNS TRIGGER AS $$  
BEGIN  
    UPDATE orders  
    SET visitor_id = NEW.new_visitor_id,  
        status = 'assigned',  
        updated_at = NOW()  
    WHERE id = NEW.order_id;  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION validate_order_status_transition()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.status = OLD.status THEN  
        RETURN NEW;  
    END IF;
```

```

    IF OLD.status = 'pending' AND NEW.status IN ('assigned', 'cancelled') THEN
        RETURN NEW;
    ELSIF OLD.status = 'assigned' AND NEW.status IN ('loading', 'cancelled')
THEN
        RETURN NEW;
    ELSIF OLD.status = 'loading' AND NEW.status IN ('delivered', 'cancelled')
THEN
        RETURN NEW;
    ELSIF OLD.status IN ('delivered', 'cancelled') THEN
        RAISE EXCEPTION 'Order status cannot change from % to %', OLD.status,
NEW.status;
    END IF;

    RAISE EXCEPTION 'Invalid order status transition from % to %', OLD.status,
NEW.status;
END;
$$ LANGUAGE plpgsql;

```

```

-- -----
-- Triggers
-- -----

```

```

CREATE TRIGGER trg_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

```

CREATE TRIGGER trg_products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

```

CREATE TRIGGER trg_cart_items_updated_at
BEFORE UPDATE ON cart_items
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

```

CREATE TRIGGER trg_orders_updated_at
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

```
CREATE TRIGGER trg_system_settings_updated_at
BEFORE UPDATE ON system_settings
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
CREATE TRIGGER trg_order_items_refresh_total_after_insert
AFTER INSERT ON order_items
FOR EACH ROW EXECUTE FUNCTION refresh_order_total_trigger();
```

```
CREATE TRIGGER trg_order_items_refresh_total_after_update
AFTER UPDATE ON order_items
FOR EACH ROW EXECUTE FUNCTION refresh_order_total_trigger();
```

```
CREATE TRIGGER trg_order_items_refresh_total_after_delete
AFTER DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION refresh_order_total_trigger();
```

```
CREATE TRIGGER trg_products_initial_history_after_insert
AFTER INSERT ON products
FOR EACH ROW EXECUTE FUNCTION insert_initial_product_history();
```

```
CREATE TRIGGER trg_order_assignments_sync_visitor_after_insert
AFTER INSERT ON order_assignments
FOR EACH ROW EXECUTE FUNCTION sync_order_visitor();
```

```
CREATE TRIGGER trg_orders_validate_status_transition
BEFORE UPDATE OF status ON orders
FOR EACH ROW EXECUTE FUNCTION validate_order_status_transition();
```

```
-- -----
-- Indexes
-- -----
```

```
CREATE INDEX idx_users_phone ON users(phone);
CREATE INDEX idx_otp_codes_phone ON otp_codes(phone);
```

```
CREATE INDEX idx_products_created_by ON products(created_by);
```

```
CREATE INDEX idx_products_is_active ON products(is_active);
CREATE INDEX idx_products_title_trgm ON products USING GIN (title
gin_trgm_ops);
```

```
CREATE INDEX idx_cart_items_user_id ON cart_items(user_id);
CREATE INDEX idx_cart_items_product_id ON cart_items(product_id);
```

```
CREATE INDEX idx_orders_buyer_id ON orders(buyer_id);
CREATE INDEX idx_orders_visitor_id ON orders(visitor_id);
CREATE INDEX idx_orders_status ON orders(status);
```

```
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_product_id ON order_items(product_id);
```

```
CREATE INDEX idx_order_assignments_order_id ON
order_assignments(order_id);
CREATE INDEX idx_order_assignments_new_visitor_id ON
order_assignments(new_visitor_id);
```

```
CREATE INDEX idx_order_status_histories_order_id ON
order_status_histories(order_id);
```

```
CREATE INDEX idx_price_histories_product_id ON price_histories(product_id);
CREATE INDEX idx_price_histories_changed_by ON
price_histories(changed_by);
```

```
CREATE INDEX idx_stock_histories_product_id ON stock_histories(product_id);
CREATE INDEX idx_stock_histories_changed_by ON
stock_histories(changed_by);
```

```
CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_notifications_related ON notifications(related_type,
related_id);
```

```
CREATE INDEX idx_account_deletion_requests_user_id ON
account_deletion_requests(user_id);
```

```
CREATE INDEX idx_account_deletion_requests_status ON
account_deletion_requests(status);
```

```
COMMIT;
```

2) seed.sql

```
-- =====
-- seed.sql
-- Initial seed data
-- =====
```

```
BEGIN;
```

```
INSERT INTO system_settings (setting_key, setting_value, description)
VALUES (
    'factory_phone',
    '09120000000',
    'Main contact number of the factory shown in product details'
)
ON CONFLICT (setting_key)
DO UPDATE SET
    setting_value = EXCLUDED.setting_value,
    description = EXCLUDED.description,
    updated_at = NOW();
```

```
COMMIT;
```

3) down.sql

```
-- =====
-- down.sql
```

-- Rollback script

-- =====

BEGIN;

DROP TABLE IF EXISTS notifications;
DROP TABLE IF EXISTS stock_histories;
DROP TABLE IF EXISTS price_histories;
DROP TABLE IF EXISTS order_status_histories;
DROP TABLE IF EXISTS order_assignments;
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS cart_items;
DROP TABLE IF EXISTS account_deletion_requests;
DROP TABLE IF EXISTS otp_codes;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS system_settings;
DROP TABLE IF EXISTS users;

DROP FUNCTION IF EXISTS validate_order_status_transition();
DROP FUNCTION IF EXISTS sync_order_visitor();
DROP FUNCTION IF EXISTS insert_initial_product_history();
DROP FUNCTION IF EXISTS refresh_order_total_trigger();
DROP FUNCTION IF EXISTS recalculate_order_total(BIGINT);
DROP FUNCTION IF EXISTS set_updated_at();

DROP TYPE IF EXISTS notification_type_enum;
DROP TYPE IF EXISTS stock_reason_enum;
DROP TYPE IF EXISTS deletion_status_enum;
DROP TYPE IF EXISTS otp_purpose_enum;
DROP TYPE IF EXISTS product_quality_enum;
DROP TYPE IF EXISTS order_status_enum;
DROP TYPE IF EXISTS user_role_enum;

DROP EXTENSION IF EXISTS pg_trgm;

COMMIT;

نکات اجرایی

- فقط یکبار هر جدول را تعریف می‌کند `schema.sql`.
 - `image_urls` می‌گیرد [] می‌گیرد، مقدار پیش‌فرض [] می‌گیرد.
 - `total_price` سفارش بعد از تغییر `order_items` دوباره محاسبه می‌شود.
 - `trigger` کنترل مسیرهای مجاز وضعیت سفارش با.
 - بعد از ثبت سفارش همچنان باید در لایه برنامه انجام شود `CartItem` حذف.
 - خوانده می‌شود `system_settings` از `factory_phone`.
-

جمع‌بندی

آماده است PostgreSQL این نسخه از نظر تکرار جدول‌ها پاک‌سازی شده و برای اجرای مستقیم در